

Query Processing

Implementation of Complex Selections

1. Introduction

Complex selection conditions combine multiple predicates using **logical operators**:

Operation	Symbol	Notation
Conjunction (AND)	$\wedge$	$\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$
Disjunction (OR)	$\vee$	$\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$
Negation (NOT)	$\neg$	$\sigma_{\neg \theta}(r)$

Key Terminology

- θ_i : Individual selection predicate (e.g., $Dept = 'IT'$)
- **RID**: Record Identifier (pointer to a specific tuple on disk)
- **Selectivity**: Fraction of tuples satisfying a predicate (lower = more selective)
- **Composite Index**: Single index on multiple attributes (e.g., $(Dept, Salary)$)

2. Conjunction: $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$

For conjunctive (AND) conditions, **all predicates must be satisfied**. Three main strategies exist:

2.1 A8: Conjunctive Selection Using One Index

Algorithm:

1. Identify the most **selective** condition θ_i that has an index.
2. Use the appropriate algorithm (A1–A6) to retrieve tuples satisfying θ_i .
3. Test remaining conditions (θ_j where $j \neq i$) **in main memory**.

Cost Estimate

$$\text{Cost} = \text{Cost of } \sigma_{\theta_i}(r) + \text{CPU time for in-memory filtering}$$

Disk I/O is determined entirely by the indexed predicate θ_i .

Example

Relation: Employee(EID, Dept, Salary)

Query: $\sigma_{\text{Dept}='IT' \wedge \text{Salary} > 50000}(\text{Employee})$

1. Assume index on **Salary** is more selective (fewer high earners than IT employees).
2. Use index to fetch all employees with $\text{Salary} > 50000$.
3. Check if $\text{Dept} = 'IT'$ in memory for each fetched tuple.

2.2 A9: Conjunctive Selection Using Composite Index

Algorithm:

1. If a **composite (multi-key) index** exists on the attributes in the conjunction, use it directly.
2. The index can filter on multiple conditions simultaneously.

Cost Estimate

$$\text{Cost} = (h_i + 1) \times (t_S + t_T) \quad (\text{for equality on all key attributes})$$

Or for range queries: $h_i \times (t_S + t_T) + t_S + b \times t_T$

Advantages:

- Reduces disk I/O since both conditions are evaluated using a single index traversal.
- Particularly effective when multiple attributes are frequently queried together.

Example

Index: Composite index on (Dept, Salary)

Query: $\sigma_{\text{Dept}='IT' \wedge \text{Salary} > 50000}(\text{Employee})$

- Navigate directly to $\text{Dept} = 'IT'$ entries in the index.
- Within those, find entries where $\text{Salary} > 50000$.
- Fetch matching records in one pass.

2.3 A10: Conjunctive Selection by Intersection of Identifiers

Algorithm:

1. For each θ_i with an available index, retrieve the set of RIDs (record pointers).
2. Compute the **intersection** of all RID sets.
3. Fetch only the records whose RIDs appear in the intersection.
4. If some conditions lack indices, apply them in memory after fetching.

Cost Estimate

$$\text{Cost} = \sum_{i=1}^k (\text{Index lookup for } \theta_i) + |\text{Intersection}| \times (t_S + t_T)$$

Where k = number of indexed conditions.

Example

Query: $\sigma_{\text{Dept}='Sales' \wedge \text{Age} < 35}(\text{Employee})$

1. Use index on **Dept** $\rightarrow$ get RIDs: $\{r_1, r_3, r_7, r_9\}$
2. Use index on **Age** $\rightarrow$ get RIDs: $\{r_2, r_3, r_5, r_9\}$
3. Intersection: $\{r_3, r_9\}$
4. Fetch only records r_3 and r_9 from disk.

3. Disjunction: $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$

For disjunctive (OR) conditions, **any predicate being true is sufficient**.

3.1 A11: Disjunctive Selection by Union of Identifiers

Algorithm:

1. For each θ_i , retrieve the set of RIDs using its index.
2. Compute the **union** of all RID sets (removing duplicates).
3. Fetch the corresponding records from the file.

Important Note

If **any condition lacks an index**, we must fall back to a **linear scan** of the entire relation. There is no way to avoid checking every tuple.

Cost Estimate

$$\text{Cost} = \sum_{i=1}^n (\text{Index lookup for } \theta_i) + |\text{Union}| \times (t_S + t_T)$$

Or if any θ_i lacks an index: $b_r \times t_T + t_S$ (linear scan).

Example

Query: $\sigma_{\text{Dept}='HR' \vee \text{Salary} > 80000}(\text{Employee})$

1. Use index on **Dept** $\rightarrow$ get HR employee RIDs.
2. Use index on **Salary** $\rightarrow$ get high-earner RIDs.
3. Union: Combine both sets, removing duplicates.
4. Fetch all records in the union set.

4. Negation: $\sigma_{\neg\theta}(r)$

Negation typically requires accessing most of the relation.

Strategies:

1. **Linear Scan:** Scan all tuples and check $\neg\theta$ for each. Most common approach.
2. **Complement via Index:** If θ is highly selective (very few matches):
 - Use the index to find RIDs satisfying θ .
 - Fetch all records **not** in this set.

Example

Query: $\sigma_{\neg(\text{Dept}='Finance')}(\text{Employee})$

- If only 5% of employees are in Finance:
 - Use index to get Finance employee RIDs.
 - Scan all blocks, skipping records with those RIDs.
- If 50% are in Finance: Just use linear scan.

5. Summary and Comparison

Algorithm	Operation	Technique	When to Use
A8	Conjunction	Single index + memory filter	One highly selective indexed condition
A9	Conjunction	Composite index	Index covers all conjunction attributes
A10	Conjunction	RID intersection	Multiple separate indices available
A11	Disjunction	RID union	All conditions have indices
–	Negation	Linear scan / complement	Default for NOT conditions

Key Takeaways

- **Conjunctions:** Prefer the most selective index, composite indices, or RID intersection.
- **Disjunctions:** Use RID union only if **all** conditions have indices; otherwise linear scan.
- **Negations:** Usually require linear scans unless the positive condition is very selective.
- **Selectivity** is the key metric for choosing which index to use first.